

**Gatien Auffret, Baptiste Cojean &
Aubin Bergeret**

ESIR 2 INFO

VV - Projet

Sommaire

Sommaire.....	2
Introduction.....	2
Description du projet.....	2
Méthodes en place.....	3
Méthodes ajoutées.....	3
Code coverage.....	3
Méthodes utilisées.....	4
Isolation et mocking (principe F.I.R.S.T).....	4
Couverture des gains rapides.....	4
Exclusion du main.....	4
Résultats obtenus.....	4
Tests d'intégration.....	5
Mutation testing.....	5
Principe.....	5
Résultats et analyse.....	5
Analyse statique.....	6
Analyse des conventions de codage (Checkstyle).....	6
Détection de bugs (SpotBugs).....	6
Conclusion et regard critique.....	8
Réalisations techniques et gain de qualité.....	8
Analyse des limites et défauts de conception.....	8
Perspectives et évolutions.....	8

Introduction

Ce projet a pour but de fiabiliser le logiciel BreizhTransport. Notre mission a consisté à transformer un simulateur existant, dépourvu de tests automatisés, en un système audité et sécurisé. Au-delà de la simple recherche de bugs, l'enjeu de ce TP a été de définir et d'appliquer une stratégie de vérification et de validation, de l'isolement des composants via le mocking jusqu'à la validation des interactions par les tests d'intégration. En croisant l'analyse dynamique avec la couverture de code, mutations et l'analyse statique, nous avons cherché à identifier les dettes techniques et les failles de conception pour garantir la robustesse du simulateur face aux évolutions futures.

Description du projet

Le projet BreizhTransport consiste en un simulateur de gestion pour un ferry de transport maritime. Ce programme gère le chargement de véhicules variés comme des voitures, des bus, des ambulances ou des cycles dans un espace aux dimensions limitées. L'application calcule les tarifs de traversée selon des règles propres à chaque catégorie de transport et gère l'organisation du pont par des algorithmes de tri.

Le système repose sur une architecture orientée objet développée en langage Java. La structure utilise des interfaces pour définir les comportements des véhicules et des classes abstraites pour les logiques de tarification et de comparaison.

Ce projet présente un intérêt pour l'étude de la vérification et de la validation logicielle car il contient des dépendances complexes. Il met en œuvre des patrons de conception comme le Singleton pour les tarifs et le Prototype pour le clonage des objets lors de l'embarquement. Le simulateur permet d'aborder des problématiques de gestion de ressources et de sécurité des données.

Méthodes en place

L'analyse du code source montre l'absence de système de gestion et d'outils de vérification et de validation dans la structure de base.

Le téléchargement des bibliothèques pour les tests ou le mocking nécessite une action manuelle.

Il n'existe pas de règle de nommage ou de formatage sur les classes du ferry.

Le lancement des tests repose sur l'exécution de classes comportant une méthode main au lieu de suites de tests autonomes.

Les commentaires ne permettent pas de vérifier la conformité du code avec les attentes.

Méthodes ajoutées

On s'organise sous une structure en pyramide pour organiser les niveaux de contrôle du logiciel. Cette organisation garantit une détection des erreurs à chaque étape du développement.

La base de la pyramide repose sur les tests unitaires. Ces vérifications isolent chaque composant, comme les tarifs ou les capacités du ferry, pour valider leur logique interne. L'usage de doublures permet de supprimer les dépendances et d'assurer une exécution rapide des tests.

Le niveau intermédiaire concerne les tests d'intégration. Cette méthode utilise les implémentations des classes pour vérifier la communication entre le ferry, les véhicules et le moteur de transport. Cette étape identifie les défauts de liaison et les problèmes de partage de données entre les modules.

Le sommet de la pyramide : les tests de mutation via l'outil PITest. Cette technique n'examine pas le code source mais la qualité de la suite de tests elle-même. Le système

injecte des fautes dans le programme pour confirmer que les assertions sont capables de détecter des régressions ou des comportements anormaux.

Code coverage

Pour mesurer notre code coverage nous avons utilisé Jacoco. Nous nous sommes d'abord fixé comme objectif minimal de 90% de coverage. L'objectif était de mesurer le pourcentage d'instructions du code source exécutées lors du lancement de la suite de tests. Ce code coverage nous permet par exemple de voir rapidement si on a oublié de tester certaines parties du code.

Méthodes utilisées

Isolation et mocking (principe F.I.R.S.T)

Pour tester la logique du Ferry, notamment ajout et tri, sans dépendre de l'implémentation réelle des véhicules, nous avons utilisé la librairie Mockito. Cela nous permet de tester le contenant indépendamment du contenu. Nous avons simulé des comportements spécifiques comme les méthodes clones(), compareTo(), pour valider les algos de tri sans effet de bord.

Couverture des gains rapides

Nous avons constaté que les classes simples comme Auto ou Bus faisaient baisser la moyenne. Nous avons donc créé une classe de test dédiée VehiculeConcretTest pour vérifier systématiquement tous les constructeurs, les méthodes toString() et les accesseurs spécifiques. Ce code est simple mais nécessite quand même d'être validé afin d'éviter les régressions bêtes.

Exclusion du main

La classe BreizhTransport sert de lanceur de simulation et non de logique métier, on a donc exclu la classe du plugin Jacoco dans le pom.xml. On fait cette exclusion car le code de simulation aléatoire fausse les résultats et n'est donc pas utile à inclure dans la mesure.

Résultats obtenus

Nous avons donc finalement réussi à obtenir un code coverage de 94% comme affiché sur la capture d'écran suivante :

projet-ferry > fr.univ.vv

fr.univ.vv

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Ferry		87 %		75 %	2	14	4	40	0	10	0	1
Tarif		87 %		n/a	1	8	2	14	1	8	0	1
ComparateurTarif		69 %		n/a	1	3	2	5	1	3	0	1
ComparateurLongueur		69 %		n/a	1	3	2	5	1	3	0	1
Ambulance		97 %		50 %	1	5	0	9	0	4	0	1
Auto		100 %		100 %	0	8	0	18	0	7	0	1
TarifBus		100 %		50 %	1	6	0	8	0	5	0	1
TarifAuto		100 %		100 %	0	6	0	8	0	5	0	1
TarifAutoTT		100 %		50 %	1	6	0	8	0	5	0	1
Vehicule		100 %		n/a	0	5	0	9	0	5	0	1
ComparateurMulti		100 %		100 %	0	3	0	8	0	2	0	1
TarifCycle		100 %		50 %	1	6	0	8	0	5	0	1
Compar		100 %		100 %	0	4	0	9	0	3	0	1
Bus		100 %		n/a	0	3	0	4	0	3	0	1
Cycle		100 %		n/a	0	4	0	7	0	4	0	1
TarifAmbulance		100 %		50 %	1	6	0	8	0	5	0	1
Total	32 of 581	94 %	7 of 26	73 %	10	90	10	168	3	77	0	16

Ce score garantit que la quasi-totalité de la logique de l'application a été exécutée au moins une fois.

Tests d'intégration

Après avoir validé notre code individuellement grâce aux tests unitaires, nous avons mis en place des tests d'intégration. Ces tests ne font pas de simulations mais utilisent de vrais objets. Le but est de vérifier que les contrats d'interface sont respectés et que les modules communiquent correctement entre eux.

Dans ce projet, nous avions l'intérêt de ces tests étaient vis à vis notamment de la cohérence physique ou de la tarification. Par exemple, mockito suppose la taille d'un véhicule alors qu'un test d'intégration intancie un vrai véhicule pour vérifier ses dimensions par exemple.

Cela nous a permis d'identifier un défaut de conception : une NullPointerException systématique due à l'absence d'auto-initialisation de la tableTarifs dans BreizhTransport.

Mutation testing

Principe

Le code coverage est très efficace pour savoir si le code a été testé mais pas pour savoir s'il a été correctement vérifié. c'est pourquoi on ajoute notamment les tests de mutations avec l'outil PiTest. Le principe consiste à générer automatiquement des variantes du code source, appelées "mutants", en introduisant de petites erreurs volontairement, par exemple changer $a < b$ en $a <= b$ ou supprimer un appel de méthode, etc...

PiTest exécute nos tests contre chaque mutant, deux résultats sont possibles, soit le mutant est tué car les tests échouent prouvant que nos tests sont suffisamment précis pour détecter l'erreur. Soit le mutant survit et donc les tests passent toujours signifiant que le code modifié n'a pas été détecté par les assertions.

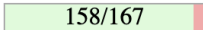
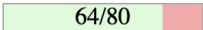
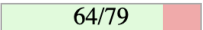
Nous avons configuré PiTest dans le pom.xml.

Résultats et analyse

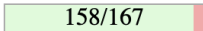
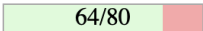
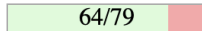
Notre analyse a produit les résultats suivants :

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
16	95%  158/167	80%  64/80	81%  64/79

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
fr.univ.vy	16	95%  158/167	80%  64/80	81%  64/79

Report generated by [PIT](#) 1.15.0

Enhanced functionality available at [arcmutate.com](#)

Le score de 80% pour Mutations Coverage indique que nos assertions sont très bonnes. Cela signifie que dans 8 cas sur 10, si une régression ou un bug est introduit dans la logique des véhicules ou du ferry, nos tests le détectent.

Analyse statique

Analyse des conventions de codage (Checkstyle)

L'analyse statique vise à uniformiser le code et à garantir sa lisibilité. Pour réaliser cette analyse, dans le pom.xml, on ajoute le plugin CheckStyle. L'objectif est de vérifier si le code respecte les standards java.

A la première exécution, le build a échoué avec plus de 500 violations de règles. Ces violations concernaient principalement le formatage avec notamment les accolades ouvrantes positionnés à la ligne suivante alors que la convention java demande l'accolade sur la même ligne que la déclaration. Une autre violation récurrente concerne la lisibilité avec l'absence d'espaces autour des opérateurs par exemple. Également certaines lignes sont trop longues et dépassent 120 caractères ce qui n'est pas une bonne pratique.

Pour forcer le build, on ajoute une ligne dans le pom.xml permettant de générer le rapport d'erreurs sans arrêter la compilation : `<failOnViolation>false</failOnViolation>`. Les erreurs bloquantes sont transformées en warnings à corriger.

Détection de bugs (SpotBugs)

En plus de Checkstyle, nous avons déployé SpotBugs pour identifier des défauts de conception, des problèmes de performance et des vulnérabilités via l'analyse du bytecode compilé. Avec `mvn spotbugs:gui`, on obtient une interface graphique rassemblant les bugs et permettant de les classer par sévérité, on affiche ci-dessous l'interface obtenue :

The screenshot shows the SpotBugs GUI. On the left, a list of bugs is displayed, including:

- Random object created and used only once in mainString()
- Auto defines clone() but doesn't implement Cloneable
- Bus defines clone() but doesn't implement Cloneable
- Cycle defines clone() but doesn't implement Cloneable
- Cycle defines compareTo() and uses Object.equals()
- Initialisation paresseuse incorrecte du champ statique BreizhTransport.tableT...
- Initialisation paresseuse incorrecte du champ statique TarifAmbulance.t dans c...
- Initialisation paresseuse incorrecte du champ statique TarifAuto.t dans createS...
- Initialisation paresseuse incorrecte du champ statique TarifBus.t dans createS...
- Initialisation paresseuse incorrecte du champ statique TarifCycle.t dans create...
- La méthode statique publique createSingleton(int, int) peut exposer une repré...
- La méthode statique publique getInstance(int, int) peut exposer une repré...
- La méthode statique publique createSingleton(int, int) peut exposer une repré...
- La méthode statique publique createSingleton(int, int) peut exposer une repré...
- La méthode statique publique getInstance(int, int) peut exposer une repré...
- La méthode statique publique getInstance(int, int) peut exposer une repré...
- La méthode statique publique getInstance(int, int) peut exposer une repré...
- La méthode statique publique getInstance(int, int) peut exposer une repré...
- La méthode newImmat(Random) concatène des chaînes au moyen de + en bouc...
- CompareurLongueur implémente Comparator mais pas Serializable
- CompareurTarif implémente Comparator mais pas Serializable

The right pane shows the source code of the `BreizhTransport` class. The bug 'Random object created and used only once' is highlighted in the code, pointing to the line:

```
28 Ferry jules = new Ferry(50 + alea.nextInt(50),
```

The bottom pane shows the bug details for 'Random object created and used only once'. It explains that the code creates a `java.util.Random` object, uses it to generate one random number, and then discards the `Random` object. This produces mediocre quality random numbers and is inefficient. It suggests using `java.security.SecureRandom` instead and avoiding allocating a new `SecureRandom` for each random number needed.

On prend pour exemple la classe `BreizhTransport` dans laquelle on repère plusieurs mauvaises pratiques. On affiche ci-dessous la page spotbug associé à cette erreur :

The screenshot shows the SpotBugs GUI. On the left, a list of bugs is displayed, including:

- Random object created and used only once in mainString()
- Auto defines clone() but doesn't implement Cloneable
- Bus defines clone() but doesn't implement Cloneable
- Cycle defines clone() but doesn't implement Cloneable
- Cycle defines compareTo() and uses Object.equals()
- Initialisation paresseuse incorrecte du champ statique BreizhTransport.tableT...
- Initialisation paresseuse incorrecte du champ statique TarifAmbulance.t dans c...
- Initialisation paresseuse incorrecte du champ statique TarifAuto.t dans createS...
- Initialisation paresseuse incorrecte du champ statique TarifBus.t dans createS...
- Initialisation paresseuse incorrecte du champ statique TarifCycle.t dans create...
- La méthode statique publique createSingleton(int, int) peut exposer une repré...
- La méthode statique publique getInstance(int, int) peut exposer une repré...
- La méthode statique publique createSingleton(int, int) peut exposer une repré...
- La méthode statique publique createSingleton(int, int) peut exposer une repré...
- La méthode statique publique getInstance(int, int) peut exposer une repré...
- La méthode statique publique getInstance(int, int) peut exposer une repré...
- La méthode statique publique getInstance(int, int) peut exposer une repré...
- La méthode statique publique getInstance(int, int) peut exposer une repré...
- La méthode newImmat(Random) concatène des chaînes au moyen de + en bouc...
- CompareurLongueur implémente Comparator mais pas Serializable
- CompareurTarif implémente Comparator mais pas Serializable

The right pane shows the source code of the `BreizhTransport` class. The bug 'Random object created and used only once' is highlighted in the code, pointing to the line:

```
28 Ferry jules = new Ferry(50 + alea.nextInt(50),
```

The bottom pane shows the bug details for 'Random object created and used only once'. It explains that the code creates a `java.util.Random` object, uses it to generate one random number, and then discards the `Random` object. This produces mediocre quality random numbers and is inefficient. It suggests using `java.security.SecureRandom` instead and avoiding allocating a new `SecureRandom` for each random number needed.

On remarque un bug de code SBSC_USE_STRINGBUFFER_CONCATENATION, il concerne la méthode newlmat() à la ligne 153. Spotbugs signale ici une concaténation de chaînes dans une boucle for. On remarque que la ligne qui pose problème est indiquée en jaune, également en bas, on remarque des indications. On a par exemple une explication sur le problème signalé et un exemple de façon de régler le souci. Ici pour régler le problème on nous propose d'utiliser un StringBuilder et .append() au lieu de concaténer deux string, cela réduira notamment la consommation mémoire.

Dans cette même méthode, on a une autre erreur qui concerne la création d'un random qui est rapidement détruit. Cela peut être très coûteux si newlmat() est appelé souvent. Aussi, si deux appels se font dans la même milliseconde, les deux instances de Random pourraient être initialisées avec la même seed (basée sur l'horloge système), générant exactement la même plaque d'immatriculation. En correction, on pourrait déclarer private static final Random ALEA = new Random(); au niveau de la classe.

Conclusion et regard critique

Le travail de vérification et de validation sur le logiciel BreizhTransport a permis d'atteindre les objectifs de couverture et de qualité fixés en début de projet. L'implémentation d'une suite de tests complets couvre les fonctionnalités de chargement, de tri et de tarification.

Réalisations techniques et gain de qualité

Maven a structuré le projet et a rendu possible l'usage d'outils d'analyse automatique. Les tests unitaires, isolés par des doublures (Mocks et Stubs), assure un bon comportement.

Le projet dispose d'indicateurs précis sur sa fiabilité :

Un code coverage de 94% garantit l'exécution de la quasi-totalité des instructions.

Un mutation coverage de 80% confirme que les assertions détectent les modifications de logique.

L'analyse statique identifie les défauts de performance et les écarts par rapport aux normes de codage.

Analyse des limites et défauts de conception

Le processus de validation montre des fragilités dans le code initial. L'usage du patron de conception Singleton pour les tarifs et la présence de méthodes statiques dans BreizhTransport freinent la testabilité du logiciel.

L'identification d'une NullPointerException lors des tests d'intégration montre que la couverture de code seule ne suffit pas à garantir la fiabilité.

Perspectives et évolutions

Pour améliorer la qualité du projet, une refonte de l'architecture vers l'injection de dépendances permettrait de supprimer les Singletons et les méthodes statiques. Cette évolution facilitera l'extension du simulateur vers de nouveaux types de véhicules ou des règles de tarification dynamiques. La prochaine étape consiste à intégrer ces tests dans une chaîne d'intégration continue pour bloquer automatiquement toute modification non conforme aux exigences de qualité.